- **Concurrent:** It is a concurrent language that supports 'communication channels' based on Hoare's Communicating Sequential Processes (CSP). The concurrency support is different from 'lock-based' programming approaches like *pthreads*, Java locks, etc.
- **Garbage-collected:** Like most modern languages, Go is garbage-collected. However, work is under way to implement 'low-latency' GC in Go.
- **Systems-programming language:** Like C, Go is a systems programming language that one can use to write things like compilers, Web servers, etc. However, we can also use it as a general-purpose programming language, for applications that create XML files, process data, etc.

Robert Griesemer, Ken Thompson (of Unix fame), and Rob Pike are the creators of the language.

Goals and motivation

Why was Go created? What are the problems that it tries to solve?

According to the creators of Go, no major systems programming language has come up in the last decade, though much has changed in the systems programming arena over the same period of time, or from even earlier. For example, libraries are becoming bigger with lots of dependencies; the Internet and networking are becoming pervasive, client-server systems and massive clusters are used today, and multi-core processors are becoming mainstream. In other words, the computing world around us is undergoing considerable change. Old systems programming languages like C and FORTRAN were not designed with these in mind, which raises the need for a new language.

Apart from these, the creators of Go felt that constructing software has become very slow. Complex and large programs have a huge number of dependencies; this makes compilation and linking painfully slow. The aim is for Go to be not just a language where we can write efficient programs, but also programs that will build quickly. Besides, object-oriented programming using inheritance hierarchies is not effective in solving problems—so the creators of Go want to have a better approach to write extensible programs.

Important characteristics

Look at some of the important features and characteristics of Go:

- **Simplicity:** Mainstream languages like C++, Java and C# are huge and bulky. In contrast, simplicity is a feature in Go's clean and concise syntax. For example, C is infamous for its complex declaration syntax. Go uses implicit type inference, with which we can avoid explicitly declaring variables. When we want to declare them, the declaration syntax is simple and convenient, and is different from C.
- **Duck typing:** Go supports a form of 'duck typing', as in many dynamic languages. A struct can automatically implement an interface—this is a powerful and novel feature of Go.
- **Goroutines:** They are not the same as threads, coroutines, or processes. Communicating between goroutines is done using a feature known as 'channels' (based on CSP), which is much safer and easier to use than the mainstream lock-based approach of *pthreads* or Java.
- **Modern features:** Go is a systems programming language, but it supports modern features like reflection, garbage collection, etc.

'Hello world' example

Here is the 'hello world' example in Go:

```
package main

import "fmt"

func main() {
  fmt.Printf("Hello world!")
}
```

All programs in Go should be in a package; it is *main* here. We import the *'fmt'* package to use its *Printf* function. Execution starts with the *main.main()* function, so we need to implement it in the program. Functions are declared using the *func* keyword. Note the lack of semicolons at the end of the lines in this program.

Looping examples

Here is a simple program that calculates the factorial of the number 5:

```
func main() {
  fact := 1
  for i := 1; i <= 5; i++ {
    fact *= i
  }
  fmt.Printf("Factorial of 5 is %d", fact)
}
```

Note that we haven't declared the variables *i* and *fact*. Since we have used the := operator, the compiler infers the type of these variables as *int* based on the initialisation value (which is '1' *for* i).

The *for* loop is the only looping construct supported in Go! If you want a *while* loop, it is a variation of the *for* loop in which the loop has only a condition check. For example:

```
fact := 1
i := 1
for i <= 5 {
  fact *= i;
  i++
}
```

Did you notice that we used a semi-colon in this code snippet? Wherever the compiler can infer semi-colons, we don't have to use them—but sometimes we do. It will take a while to get used to the rules about semicolons in Go.